

nivel 1, la memoria total necesaria se multiplicaría por ocho. Además, la memoria necesaria sería memoria de escritura para control, que es muchísimo más cara debido a su gran velocidad. No conviene usar la memoria principal para el microcódigo, ya que obtendríamos una máquina lenta.

A la luz de estos ejemplos concretos, se advertirá claramente por qué las máquinas se diseñan como una serie de niveles. Se hace por razones de eficiencia y simplicidad, ya que cada nivel se aplica a un nivel de abstracción distinto. El diseñador del nivel 0 se ocupa de cómo ganar unos pocos nanosegundos en la ALU utilizando algún nuevo algoritmo que reduzca el tiempo de propagación del acarreo. El microprogramador se interesa en cómo hacer el máximo número de operaciones elementales con cada microinstrucción, aprovechando en lo posible el paralelismo inherente al hardware. El diseñador del juego de macroinstrucciones desea proporcionar al escritor de compiladores y al microprogramador una interfaz con la que se puedan sentir a gusto y que, al mismo tiempo, sea eficiente. Sin duda cada nivel tiene objetivos, problemas y técnicas diferentes y, en general, una forma distinta de ver la máquina. Dividiendo el problema del diseño de la máquina en varios subproblemas, podemos intentar dominar la complejidad intrínseca del diseño de una computadora moderna.

4.5. EL DISEÑO DEL NIVEL DE MICROPROGRAMACION

Como cualquier otra cosa en informática, el diseño de la microarquitectura está lleno de limitaciones. En las secciones siguientes veremos algunos de los temas de diseño y los problemas que han de sopesarse.

4.5.1. Microprogramación horizontal frente a microprogramación vertical

Probablemente la decisión más importante sea cuán codificadas deban estar las microinstrucciones. Si fuéramos a construir la Mic-1 con una sola pastilla VLSI, se podrían ignorar abstracciones tales como registros, ALU, etc., y pensar sólo en las compuertas. Para que funcione la máquina, se necesitan ciertas señales, como las 16 de habilitación de salida de los registros al bus A o las que controlan el funcionamiento de la ALU. Si miráramos dentro de la ALU, veríamos que toda la circuitería interna está controlada por cuatro líneas, no dos, ya que en la esquina izquierda de la figura 3-20 encontramos un decodificador de 2 a 4. En resumen, se puede hacer funcionar cualquier máquina con n señales de control aplicadas en los lugares adecuados sin decodificar nada.

Este punto de vista nos obliga a considerar un formato de microinstrucción diferente: hacerlo de n bits, uno por señal de control. Las microinstrucciones diseñadas según este principio se denominan **horizontales** y representan un extremo del espectro de posibilidades. En el otro extremo están las microinstrucciones con un pequeño número de campos muy codificados. Se dice que

son **verticales**. Los nombres derivan de la manera en que un artista dibujaría sus memorias de control respectivas: los diseños horizontales tienen un número bastante pequeño de microinstrucciones anchas; los verticales tienen muchas microinstrucciones estrechas.

Entre ambos extremos hay muchos diseños intermedios. Nuestras microinstrucciones, por ejemplo, tienen algunos bits, como MAR, MBR, RD, WR y AMUX, que controlan directamente funciones del hardware. Por otro lado, los campos A, B, C y ALU requieren alguna lógica de decodificación antes de que puedan aplicarse a compuertas individuales. Una instrucción vertical extrema tendría solamente un código de operación, que sería simplemente una generalización de nuestro campo ALU, y algunos operandos, como nuestros campos A, B y C. En una organización de este tipo se necesitarían códigos de operación para leer y escribir en memoria, hacer microsaltos, etc., porque los campos que controlan esas funciones en nuestra máquina ya no estarían presentes.

Para distinguir más claramente entre microinstrucciones horizontales y verticales, vamos a rediseñar nuestro ejemplo de microarquitectura y hacer que se use microinstrucciones verticales. Cada microinstrucción contendrá ahora tres campos de 4 bits, que suman un total de 12 bits, frente a los 32 de la versión original. El primer campo es el código de operación, OP, que dice qué hace la microinstrucción. Los siguientes campos son dos registros, R1 y R2. Para los saltos, se combinan formando un único campo de 8 bits, R. Una microinstrucción típica sería ADD, SP, AC, que sumará el AC al SP.

En la figura 4-17 se muestra la lista completa de códigos de operación de las microinstrucciones de esta nueva máquina que llamaremos Mic-2. En la lista vemos que cada microinstrucción realiza una única función: si suma, no puede desplazar ni cargar el MAR, ni siquiera mantener activa la señal RD. Con apenas 12 bits por microinstrucción, sólo hay sitio para especificar una operación.

Vamos a rehacer la figura 4-10 para mostrar las nuevas microinstrucciones. La figura 4-18 muestra el nuevo diagrama de bloques. La ruta de datos, representada a la izquierda, es idéntica a la anterior. La mayor parte de la porción de control, a la derecha, también quedará igual. En particular, aún necesitamos el MIR y la memoria de control (aunque esta vez con anchura de 12 bits, en lugar de 32). Los tamaños y funciones del MPC, el Mmux, el incrementador, el reloj y la lógica de microsecuenciamiento son idénticos a los del diseño horizontal. Además, necesitaremos decodificadores de 4 a 16 para los campos R1 y R2, análogos a los de A, B y C de la figura 4-10.

Las tres principales diferencias entre la figura 4-10 y la figura 4-18 son los bloques rotulados AND, NZ y "Decodificación de OP". Se necesita AND porque el campo R1 lleva tanto el bus A como el C. Se presenta el problema de que el bus A se carga durante el subciclo 2 pero el bus C no puede cargarse en la memoria interna hasta que se hayan estabilizado los registros de A y B, en el subciclo 3.

Binario Nemotécnico	Instrucción	Significado
0000	ADD	$r1 := r1 + r2$
0001	AND	$r1 := r1 \text{ Y } r2$
0010	MOVE	Mueve registro $r1 := r2$
0011	COMPL	Complementa $r1 := \text{inv}(r2)$
0100	LSHIFT	Desplaza a la izquierda $r1 := \text{desizq}(r2)$
0101	RSHIFT	Desplaza a la derecha $r1 := \text{desder}(r2)$
0110	GETMBR	Almacena el MBR en registro $r1 := \text{rim}$ — MBR
0111	TEST	Examina registro $\text{if } r2 < 0 \text{ then } n := \text{true}; \text{if } r2 = 0 \text{ then } z := \text{true}$
1000	BEGRD	Comienza lectura $\text{rdm} := r1; \text{lec}$ — MAR = rdm
1001	BEGWR	Comienza escritura $\text{rdm} := r1; \text{rim} := r2; \text{esc}$
1010	CONRD	Continúa lectura lec
1011	CONWR	Continúa escritura esc
1100		(no usado)
1101	NJUMP	Salta si N = 1 $\text{if } n \text{ then goto } r$
1110	ZJUMP	Salta si Z = 1 $\text{if } z \text{ then goto } r$
1111	UJUMP	Salta siempre $\text{goto } r$

$$r = 16 * r1 + r2$$

Fig. 4-17. Códigos de operación del Mic-2.

La caja AND hace el Y lógico de cada una de las 16 señales decodificadas con la señal del reloj del subciclo 4 y con una señal que proviene de la decodificación de OP y que equivale a la vieja señal ENC. El resultado es que las 16 señales que cargan datos en la memoria interna se activan bajo las mismas condiciones que antes.

El bloque NZ es un registro de dos bits al que se le pueden hacer almacenar las señales N y Z del ALU. Se necesita esta facilidad ya que en el nuevo diseño, el ALU trabaja en una microinstrucción pero sólo puede verificar los bits de estado en la siguiente. Como la ALU no tiene donde almacenar N y Z y éstas se obtienen de su salida en cada momento, siendo N su bit más significativo y Z el 0 lógico de todos sus bits, ambas señales de estado se perderían si no se guardarán en alguna parte.

El elemento clave de la nueva microarquitectura es el decodificador de OP. Esta caja toma el campo de código de operación y produce señales para controlar la caja AND, la lógica de microsequeñamiento, NZ, Amux, la ALU, el registro de corrimiento, el MBR, el MAR, RD y WR. La lógica de microsequeñ-

